Residência em Microeletrônica — Polo UFCG / CI-Expert

Documento de Microarquitetura (v1.0)

MiniSoC AES — Especificação Interna de Implementação RTL

Autor: Daniel Lima Neto
Trilha de Projeto RTL & Verificação Funcional

Novembro de 2026

Resumo

Este documento especifica a microarquitetura interna do **MiniSoC AES** a partir das interfaces de topo, mapa de registradores e protocolo SPI fixados no *Block Guide*. São definidos: (1) o diagrama de blocos interno e o isolamento da memória SRAM *single-port*; (2) a microarquitetura iterativa de 128 bits (1 rodada/ciclo) do núcleo AES e a expansão pré-calculada de chaves; (3) a máquina de estados finitos (FSM) do controlador `minisoc_aes_top`; (4) a travessia de domínios de relógio (CDC) entre `spi_sclk` e `clk` por sincronização *toggle*; e (5) a sequência e sincronização de reset do domínio `sys_clk` condicionada ao *lock* do PLL.

---

1 Diagrama de Blocos Interno e Interconexões

O módulo de topo (`minisoc_top.sv`) integra quatro blocos funcionais principais (`minisoc_spi`, `minisoc_pll_mock`, `minisoc_aes_top` e `minisoc_memory`), além do sincronizador de reset do domínio do sistema e do multiplexador de arbitragem da memória compartilhada, conforme ilustrado na Figura 1.

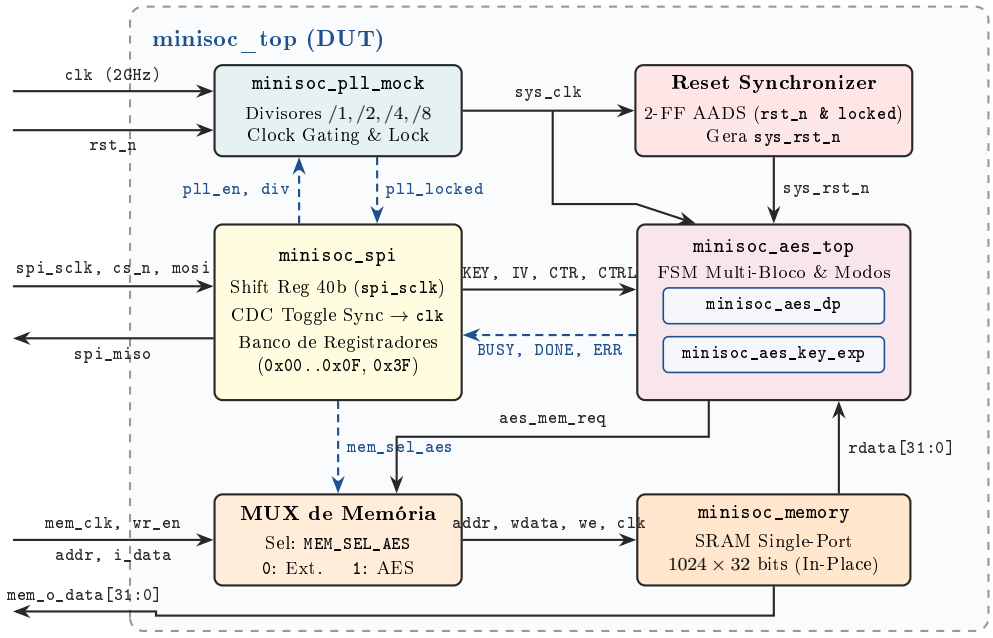


Figura 1: Diagrama de blocos interno do `minisoc_top` e arbitragem da SRAM.

1.1 Isolamento e Arbitragem da Memória Compartilhada

Como a SRAM `minisoc_memory` (1024 × 32 bits) é *single-port*, o acesso é multiplexado pelo bit `MEM_SEL_AES` do registrador `SYSTEM_CONTROL`:

- `MEM_SEL_AES = 0` **(Acesso Externo):** Os sinais externos `mem_clk`, `mem_i_wr_en`, `mem_i_addr` e `mem_i_data` controlam diretamente a SRAM para carga de dados e leitura de resultados.
- `MEM_SEL_AES = 1` **(Acesso Exclusivo do AES):** A porta da memória é dedicada à FSM do `minisoc_aes_top` sob o relógio `sys_clk`, bloqueando escritas externas concorrentes durante o processamento *in-place*.

2 Microarquitetura do Núcleo Criptográfico AES

2.1 Justificativa da Arquitetura Iterativa de 128 bits (1 Rodada/Ciclo)

A Tabela 1 compara as três alternativas clássicas de microarquitetura para o datapath `minisoc_aes_dp` diante das restrições do *Block Guide*.

Tabela 1: Comparação de arquiteturas para o datapath AES-128 no MiniSoC.

Arquitetura	**Área (S-Boxes)**	**Latência/Bloco**	**Acoplamento com SRAM 32b e Modos**	**Decisão**
Pipelined (10 estágios)	Alta (≈ 160)	1 ciclo (após fill)	Gargalo de I/O (8 ciclos/bloco) e trava em CBC/CFB	Descartada
Seriada (8 ou 32 bits)	Baixa (1 a 4)	44 a 160 ciclos	Controle complexo de `ShiftRows` e alta latência	Descartada
Iterativa 128b (1 rod./ciclo)	**Média** ($16+16$)	**10 ciclos**	**Casada à SRAM (8 ciclos I/O) e modos encadeados**	**Adotada**

A arquitetura **iterativa de 128 bits com 1 rodada por ciclo** foi escolhida por dois motivos determinantes:

1. **Largura de Banda da Memória e Dependência Entre Blocos:** Como cada bloco de 128 bits requer 4 ciclos de leitura e 4 ciclos de escrita na SRAM de 32 bits, e os modos encadeados (`CBC`, `PCBC`, `CFB`, `OFB`) exigem o término do bloco i antes de iniciar o bloco $i+1$, um pipeline de 10 estágios ficaria ocioso.
2. **Simplicidade e Modularidade RTL:** Opera diretamente sobre o estado completo de 128 bits (`state_reg[127:0]`), conectando diretamente os módulos combinacionais da FIPS-197 sem multiplexação sub-palavra.

2.2 Submódulos de Transformação de Rodada (`minisoc_aes_dp`)

Seguindo a convenção de estado em que o byte 0 ocupa `[127:120]` e a coluna $c \in \{0..3\}$ ocupa `state[127-32*c -: 32]`, o datapath instancia:

- **SubBytes e InvSubBytes (`minisoc_aes_sub_bytes`, `minisoc_aes_inv_sub_bytes`):** Cada módulo instancia 16 tabelas combinacionais de 256 entradas (`minisoc_aes_sbox` da Figura 7 e `minisoc_aes_inv_sbox` da Figura 14 da FIPS-197).
- **ShiftRows e InvShiftRows (`minisoc_aes_shift_rows`, `minisoc_aes_inv_shift_rows`):** Reordenação pura de fios que desloca ciclicamente a linha r em r bytes para a esquerda (cifragem) ou para a direita (decriptação).
- **MixColumns e InvMixColumns (`minisoc_aes_mix_columns`, `minisoc_aes_inv_mix_columns`):** Multiplicação matricial em $GF(2^8)$ com polinômio redutor `8'h1b` (coeficientes $\{02, 03, 01, 01\}$ via função `xtime` na direta, e $\{0e, 0b, 0d, 09\}$ na inversa). Na 10ª rodada, a saída de `ShiftRows`/`InvShiftRows` desvia desta etapa via multiplexador.
- **AddRoundKey (`minisoc_aes_add_round_key`):** Porta XOR de 128 bits entre o estado e a subchave da rodada.

2.3 Expansão de Chave Pré-Calculada (`minisoc_aes_key_exp`)

Nos modos `ECB`, `CBC` e `PCBC`, a decriptação utiliza o *Inverse Cipher*, que exige as subchaves na ordem reversa ($K_{10} \rightarrow K_0$). Para evitar recalcular até K_{10} a cada bloco, o módulo `minisoc_aes_key_exp` pré-calcula e armazena todas as 11 subchaves (`round_key[0..10]`) em 10 ciclos no início da operação (estado `KEY_EXP`), amortizando esse custo para lotes de até 256 blocos.

3 Máquina de Estados Finita (FSM) do `minisoc_aes_top`

A Figura 2 apresenta a FSM síncrona ao relógio `sys_clk` que coordena a validação de parâmetros, a expansão de chave, os acessos *in-place* à SRAM e o datapath.

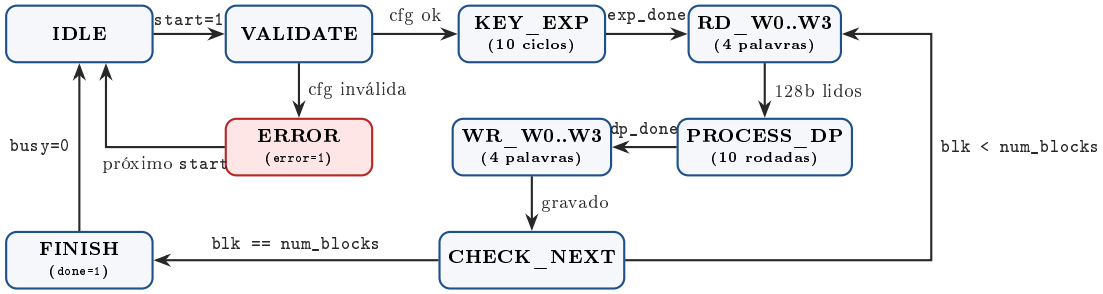


Figura 2: Diagrama de estados da FSM de controle no `minisoc_aes_top`.

- **`IDLE` → `VALIDATE`:** Ao detectar `aes_start`, captura a configuração e verifica se $1 \leq$ `NUM_BLOCKS` ≤ 256 e `MODE` $\in \{0..5\}$. Se inválida, transita para `ERROR` (`aes_error = 1`, `aes_busy = 0`).
- **`KEY_EXP`:** Executa 10 ciclos para preencher `round_key[0..10]`.
- **`RD_W0..W3` e `WR_W0..W3`:** Lê as 4 palavras de 32 bits do bloco i nos endereços $(4i + 0..3)$ da SRAM e, após os 10 ciclos de `PROCESS_DP`, reescreve o bloco processado nos mesmos 4 endereços.
- **`CHECK_NEXT` e `FINISH`:** Incrementa `blocks_done`; se restarem blocos, retorna a `RD_W0..W3`; caso contrário, sinaliza `aes_done = 1` (*sticky*) e retorna a `IDLE`.

4 Estratégia de Clock-Domain Crossing (CDC)

A fronteira assíncrona ocorre entre `spi_sclk` (≤ 25 MHz, $T \geq 40$ ns) e o relógio *always-on* `clk` (2 GHz, $T = 0{,}5$ ns). Conforme a Seção 5.3 do *Block Guide*, utiliza-se um **sincronizador por *toggle*** aliado a um registrador de captura estático de 40 bits (Figura 3).

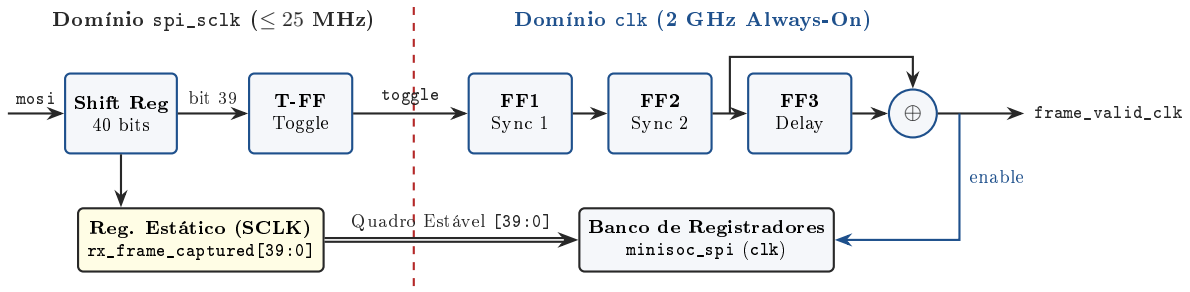


Figura 3: Circuito CDC por *Toggle* e captura estática do quadro de 40 bits.

Ao receber o 40º bit em `spi_sclk`, o quadro é travado em `rx_frame_captured[39:0]` e o sinal `toggle` inverte de nível. Após atravessar os 2 estágios sincronizadores (`FF1`, `FF2`) e o detector de borda (`FF2` $\oplus$ `FF3`), o pulso de 1 ciclo `frame_valid_clk` habilita a decodificação e escrita no domínio `clk`. Como o próximo quadro SPI leva ao menos 40 ciclos de `spi_sclk` (≥ 3200 ciclos de `clk`), o barramento `rx_frame_captured` permanece estático durante toda a amostragem.

5 Sequência e Sincronização de Reset do Domínio `sys_clk`

Conforme a Seção 4.2 do *Block Guide*, o relógio `sys_clk` permanece em nível 0 enquanto o PLL não estiver travado (`pll_locked = 0`). Para garantir que o núcleo AES seja inicializado em estado conhecido mesmo sem relógio oscilante e saia do reset sem violação de *recovery/removal*, emprega-se um sincronizador **AADS (*Asynchronous Assert, Synchronous Deassert*)** de 2 estágios (Figura 4).

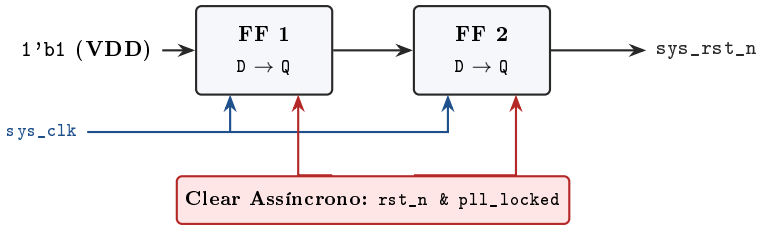


Figura 4: Sincronizador de reset AADS para o domínio `sys_clk`.

Sequência de inicialização: (1) `rst_n` sobe para 1 e libera a interface SPI no domínio `clk`; (2) o mestre SPI programa `SYSTEM_CONTROL` com `PLL_ENABLE = 1` e `PLL_DIV_SEL`; (3) após o tempo de aquisição, `pll_locked` sobe para 1 e libera `sys_clk`; (4) o clear assíncrono (`rst_n & pll_locked`) é liberado e, após duas bordas de subida de `sys_clk`, `sys_rst_n` sobe sincronamente para 1.